

TRIGGERS EN SQL SERVER

Tabla de contenidos

1. ¿Qué es un trigger?.....	2
1.1 Flujo de ejecución	2
2. Tipos de triggers en SQL Server	2
2.1 Triggers DML (sobre datos)	2
2.2 Triggers DDL (sobre estructura).....	2
2.3 Triggers LOGON (sobre conexiones)	2
2.4 AFTER / FOR vs INSTEAD OF (en triggers DML).....	2
3. Las tablas inserted y deleted	3
4. Ventajas y desventajas.....	3
5. Ejemplos prácticos	3
5.1 Validación con INSTEAD OF INSERT.....	3
5.2 Auditoría con AFTER DELETE.....	4
5.3 Control de precio con AFTER UPDATE	4
6. Cosas importantes a considerar	5
6.1 Los triggers son parte de la transacción.....	5
6.2 INSTEAD OF vs AFTER: diferencia crítica en COUNT	5
6.3 Siempre usar SET NOCOUNT ON.....	5
6.4 Habilitar y deshabilitar triggers	5
6.5 Rendimiento.....	6
7. Resumen rápido	6

1. ¿Qué es un trigger?

Un trigger (disparador) es un procedimiento almacenado especial que SQL Server ejecuta automáticamente cuando ocurre un determinado evento sobre una tabla, vista o la base de datos en general.

A diferencia de los procedimientos almacenados comunes, un trigger no se invoca manualmente: se dispara solo ante el evento que lo activa, sin que el código cliente deba hacer nada adicional.

1.1 Flujo de ejecución

El siguiente esquema muestra qué ocurre cuando se ejecuta una operación que tiene un trigger asociado:



2. Tipos de triggers en SQL Server

2.1 Triggers DML (sobre datos)

Son los más utilizados. Se activan ante operaciones de manipulación de datos: INSERT, UPDATE o DELETE sobre tablas o vistas.

2.2 Triggers DDL (sobre estructura)

Se activan ante cambios en la estructura de la base de datos: CREATE TABLE, DROP TABLE, ALTER TABLE, etc. Son útiles para registrar quién modificó el esquema y cuándo.

2.3 Triggers LOGON (sobre conexiones)

Se activan cuando un usuario inicia sesión en la instancia de SQL Server. Permiten controlar o restringir el acceso según condiciones específicas.

2.4 AFTER / FOR vs INSTEAD OF (en triggers DML)

Dentro de los triggers DML, SQL Server ofrece dos momentos de ejecución con comportamientos muy distintos:

Característica	AFTER (o FOR)	INSTEAD OF
Cuándo se ejecuta	Después que la operación ya ocurrió en la tabla	En lugar de la operación; el programador decide si la ejecuta o no
¿La fila ya está en la tabla?	Sí, ya fue insertada/modificada/eliminada	No todavía — el trigger decide si la inserta
¿Puede cancelar la operación?	Sí, con ROLLBACK TRANSACTION	Sí, simplemente no ejecutando el INSERT/UPDATE/DELETE
Uso más común	Auditoría, log de cambios históricos	Validaciones antes de insertar, vistas actualizables

3. Las tablas inserted y deleted

Dentro de cualquier trigger DML, SQL Server pone a disposición dos tablas temporales especiales que contienen los datos afectados por la operación que disparó el trigger:

Tabla inserted	Tabla deleted
- En un INSERT: contiene las filas que se están insertando. - En un UPDATE: contiene los valores nuevos de las filas modificadas. - Su estructura es idéntica a la tabla original.	- En un DELETE: contiene las filas que se están eliminando. - En un UPDATE: contiene los valores anteriores de las filas modificadas. - Su estructura es idéntica a la tabla original.

Resumen por tipo de operación:

Operación	inserted contiene	deleted contiene
INSERT	Las filas nuevas que se insertan	Vacía
DELETE	Vacía	Las filas que se eliminan
UPDATE	Los valores nuevos (después del cambio)	Los valores anteriores (antes del cambio)

4. Ventajas y desventajas

Ventajas	Desventajas
- La lógica queda centralizada en la base de datos, independiente de la aplicación cliente. - Se ejecutan automáticamente, sin intervención del código que hace el INSERT/UPDATE/DELETE. - Ideales para auditoría: registran quién cambió qué y cuándo. - Pueden implementar validaciones de integridad complejas que las restricciones (CONSTRAINTS) no permiten. - Son transparentes para la aplicación que ejecuta la operación.	- Dificultan el diagnóstico: la lógica 'está escondida' en la base de datos. - Impactan el rendimiento si contienen lógica compleja o consultas costosas. - Pueden encadenarse (trigger que dispara otro trigger), generando comportamientos difíciles de trazar. - Un error no manejado dentro del trigger puede cancelar toda la transacción del cliente. - No se pueden desactivar fácilmente desde el código de la aplicación.

5. Ejemplos prácticos

5.1 Validación con INSTEAD OF INSERT

Evita insertar una marca duplicada. Con INSTEAD OF, la fila aún no está en la tabla cuando el trigger corre; por eso EXISTS detecta correctamente los duplicados.

Nota: se usa JOIN con inserted en lugar de variables escalares para soportar correctamente la inserción de múltiples filas a la vez.

```

CREATE OR ALTER TRIGGER tr_Marca
ON Marcas
INSTEAD OF INSERT
AS
BEGIN
    SET NOCOUNT ON;

    -- Verificar si alguna de las marcas a insertar ya existe
    IF EXISTS (
        SELECT 1 FROM Marcas m
        INNER JOIN inserted i ON m.Marca = i.Marca
    )
    BEGIN
        RAISERROR('La marca ya existe en la tabla.', 16, 1);
    END
    ELSE
    BEGIN
        -- La fila aún no existe: la insertamos nosotros
        INSERT INTO Marcas (Marca, PaisID)
        SELECT Marca, PaisID FROM inserted;
    END
END

```

5.2 Auditoría con AFTER DELETE

Registra automáticamente en una tabla de auditoría cada eliminación de producto, incluyendo el usuario y la fecha y hora del evento.

```

-- Tabla de auditoría requerida:
-- Auditoria (Tabla, Accion, IDAfectado, Usuario, Fecha)

CREATE OR ALTER TRIGGER tr_AuditarEliminacion
ON Productos
AFTER DELETE
AS
BEGIN
    SET NOCOUNT ON;

    -- Insertar un registro por cada fila eliminada
    INSERT INTO Auditoria (Tabla, Accion, IDAfectado, Usuario, Fecha)
    SELECT
        'Productos',
        'DELETE',
        d.ProductoID,
        SYSTEM_USER,
        GETDATE()
    FROM deleted d;
END

```

5.3 Control de precio con AFTER UPDATE

Impide que el precio de un producto sea reducido. Compara los valores anteriores (tabla deleted) con los nuevos (tabla inserted) y aplica ROLLBACK si detecta una reducción.

```

CREATE OR ALTER TRIGGER tr_ControlPrecio
ON Productos
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    -- Comparar precio nuevo vs precio anterior
    IF EXISTS (
        SELECT 1
        FROM inserted i
        INNER JOIN deleted d ON i.ProductoID = d.ProductoID
        WHERE i.Precio < d.Precio
    )
    BEGIN
        ROLLBACK TRANSACTION;
        RAISERROR('No se puede reducir el precio de un producto.', 16,
1);
    END
END

```

6. Cosas importantes a considerar

6.1 Los triggers son parte de la transacción

Un trigger DML se ejecuta dentro de la misma transacción que lo disparó. Si el trigger hace ROLLBACK TRANSACTION, también se deshace la operación original (el INSERT, UPDATE o DELETE del cliente).

6.2 INSTEAD OF vs AFTER: diferencia crítica en COUNT

En un trigger INSTEAD OF INSERT, la fila nueva NO está todavía en la tabla al momento de ejecutarse el trigger. Por eso, para verificar duplicados, la condición correcta es EXISTS (no COUNT(*) > 1). En un trigger AFTER INSERT, la fila YA está en la tabla.

6.3 Siempre usar SET NOCOUNT ON

Incluir SET NOCOUNT ON al inicio del trigger para suprimir los mensajes de 'N filas afectadas' de las operaciones internas. Sin esto, la aplicación cliente puede recibir mensajes confusos o el trigger puede generar errores en aplicaciones que cuentan los rowcount.

6.4 Habilitar y deshabilitar triggers

Es posible habilitar o deshabilitar un trigger sin necesidad de eliminarlo. Esto es útil para cargas masivas de datos o mantenimiento:

```

-- Deshabilitar un trigger específico
DISABLE TRIGGER tr_Marca ON Marcas;

-- Habilitar nuevamente
ENABLE TRIGGER tr_Marca ON Marcas;

-- Deshabilitar todos los triggers de una tabla
DISABLE TRIGGER ALL ON Marcas;

```

6.5 Rendimiento

Evitar lógica de negocios compleja, cursores o consultas costosas dentro de triggers: se ejecutan en cada operación DML que los dispare, y pueden degradar significativamente el rendimiento en tablas con alta concurrencia.

7. Resumen rápido

Concepto	Definición resumida
Trigger	Procedimiento que se ejecuta automáticamente ante un evento DML, DDL o LOGON
AFTER	Se ejecuta después de que la operación ya ocurrió en la tabla
INSTEAD OF	Reemplaza la operación; el trigger decide si ejecutarla o no
inserted	Tabla temporal con las filas nuevas (INSERT) o valores nuevos (UPDATE)
deleted	Tabla temporal con las filas eliminadas (DELETE) o valores anteriores (UPDATE)
RAISERROR	Lanza un error de SQL Server con un mensaje y severidad definidos
ROLLBACK	Cancela la transacción entera, incluyendo la operación original